

1 TypeSpec Syntax Specification

Annotations	A	::=	$\overline{\text{@spec } E}$
Expressions	E	::=	$x(\bar{T}) :: T \mid x(\bar{T}) :: T \text{ when } \overline{\alpha : \bar{T}}$
Types	T	::=	$\alpha \mid T \mid T$ $\mid \text{any}() \mid \text{none}()$ $\mid \text{atom}() \mid :k$ $\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{float}() \mid \text{integer}()$ $\mid \text{neg_integer}() \mid \text{non_neg_integer}() \mid \text{pos_integer}()$ $\mid \text{tuple}() \mid \{\bar{T}\} \mid \% \{\bar{F}\}$ $\mid [] \mid \text{nonempty_maybe_improper_list}(T, T)$ $\mid \ll \mid \ll :: n \gg \mid \ll :: *n \gg \mid \ll :: n, _ :: *n \gg$ $\mid (\bar{T} \rightarrow T) \mid (\dots \rightarrow T)$ $\mid n \mid n..n'$
Fields	F	::=	$\text{required}(T) \Rightarrow T \mid \text{optional}(T) \Rightarrow T$

where x ranges over expression variables, α ranges over type variables, n ranges over integers with $n < n'$, and k ranges over atom names (N.B., the notation $\overline{E}$ denotes a possibly empty sequence of elements E separated by commas apart from $\overline{\text{@spec } E}$ and $\overline{\alpha : \bar{T}}$, where it denotes a non-empty sequence.)

Figure 1: TypeSpec

1.1 TypeSpec Built-in Types and Syntactic Sugar

Built-in types	Defined as
term()	any()
arity()	0..255
*as_boolean(T)	T
binary()	«_::_*8»
nonempty_binary()	«_::_8, _::_*8»
bitstring()	«_::_*1»
nonempty_bitstring()	«_::_1, _::_*1»
boolean()	:true :false
byte()	0..255
nonempty_list(T)	nonempty_maybe_improper_list(T, [])
list(T)	[] nonempty_list(T)
nonempty_list()	nonempty_list(any())
*nonempty_improper_list(T ₁ , T ₂)	nonempty_maybe_improper_list(T ₁ , T ₂)
maybe_improper_list(T ₁ , T ₂)	[] nonempty_improper_list(T ₁ , T ₂)
maybe_improper_list()	maybe_improper_list(any(), any())
nonempty_maybe_improper_list()	nonempty_maybe_improper_list(any(), any())
char()	0..0x10FFFF
charlist()	list(char())
nonempty_charlist()	nonempty_list(char())
fun()	(... -> any())
function()	fun()
identifier()	pid() port() reference()
iodata()	iolist() binary()
iolist()	maybe_improper_list(byte() binary() iolist(), binary() [])
keyword()	[{atom(), any()}]
keyword(T)	[{atom(), T}]
mfa()	{module(), atom(), arity()}
module()	atom()
no_return()	none()
node()	atom()
number()	integer() float()
struct()	%{:_struct__ => atom(), optional(atom()) => any()}
timeout()	:infinity non_neg_integer()

(built-in types with prepending asterisk (*) are only listed for documentation)

Table 1: TypeSpec Built-in Types

Syntactic sugar	Expanded as
true	:true
false	:false
nil	:nil
[T, ...]	nonempty_list(T)
[T]	list(T)
T ₁ => T ₂	required(T ₁) => T ₂
k:T	:k => T
%k{F}	%{:__struct__ => :k, F}

Table 2: TypeSpec Syntactic Sugars

2 Elixir Types Syntax Specification

Annotations	A ::= \$S \$S when $\overline{\alpha:T}$	
Signatures	S ::= $\overline{T} \rightarrow T$ S and S	
Types	T ::= α K term() none() dynamic() T or T T and T not T n..n infty..n n..infty { $\overline{T}$ } { $\overline{T}, \dots$ } %{ $\overline{F}$ } %{ $\dots, \overline{F}$ } ($\overline{T} \rightarrow T$) empty_list() non_empty_list(T, T)	type variables basic types type connectives integers (internal representation) type constructors
Key types	K ::= atom() :k pid() port() reference() float() integer() binary() tuple() open_map() fun() list()	
Fields	F ::= :k=>T K=>if_set(T)	

where x ranges over expression variables, α ranges over type variables, n ranges over integers (integers internal representation: types not available to the programmer), and k ranges over atom names. (N.B., the notation $\overline{E}$ denotes a possibly empty sequence of elements E separated by commas apart from { $\overline{T}, \dots$ }, %{ $\dots, \overline{F}$ }, and $\overline{\alpha:T}$, where it denotes a non-empty sequence. Also, in $\overline{F}$ each F must have a different keytype)

Figure 2: Elixir Types Syntax

2.1 Elixir Type Built-in Types and Syntactic Sugar

Built-in types	Defined as
boolean()	:true or :false
non_empty_list(T)	non_empty_list(T, empty_list())
list(T, T')	empty_list() or non_empty_list(T, T')
list(T)	empty_list() or non_empty_list(T, T')
list()	empty_list() or non_empty_list(term(), empty_list())

Table 3: Elixir Type Built-in Types

Syntactic sugar	Expanded as
$\{\dots\}$	<code>tuple()</code>
$\%\{\dots\}$	<code>open_map()</code>
$k : T$	<code>:k => T</code>
<code>K => not_set()</code>	<code>K => if_set(none())</code>
<code>K => T</code> <i>(for $K \neq :k$)</i>	<code>K => if_set(T)</code>

Table 4: Elixir Type Syntactic Sugars

3 Translation from TypeSpec to Elixir Types

$\llbracket \cdot \rrbracket : \text{TypeSpec} \rightarrow \text{Elixir type}$

Figure 3: Semantic of Translation

3.1 Translation of TypeSpec Specification into Elixir Annotations

$\llbracket @spec E \rrbracket = \$ \llbracket E \rrbracket$

$\llbracket @spec \bar{E}, @spec E' \rrbracket = \$ \llbracket \bar{E}, E' \rrbracket$

$\llbracket x(\bar{T}_1) :: T_2 \rrbracket = \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket$

$\llbracket x(\bar{T}_1) :: T_2 \text{ when } \bar{\alpha} : \bar{T}_3 \rrbracket = \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket$

$\llbracket \bar{E}, x(\bar{T}'_1) :: T'_2 \rrbracket = \begin{cases} S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ \llbracket \bar{E} \rrbracket \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket & \text{otherwise} \end{cases}$

$\llbracket \bar{E}, x(\bar{T}'_1) :: T'_2 \text{ when } \bar{\alpha}' : \bar{T}'_3 \rrbracket = \begin{cases} S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket, \bar{\alpha}' : \llbracket \bar{T}'_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ & \text{and } \bar{\alpha} \cap \bar{\alpha}' = \emptyset \\ S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket[\alpha' := \beta'] \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket, \bar{\beta}' : \llbracket \bar{T}'_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ & \text{and } \bar{\alpha} \cap \bar{\alpha}' \neq \emptyset \\ \llbracket \bar{E} \rrbracket \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha}' : \llbracket \bar{T}'_3 \rrbracket & \text{otherwise} \end{cases}$

Figure 4: Translation into Annotations

3.2 Translation of TypeSpec into Elixir Types

```

[[α]] = α
[[T1 | T2]] = [[T1]] or [[T2]]
[[any()]] = term()
[[none()]] = none()
[[atom()]] = atom()
[[ :k ]] = :k
[[pid()]] = pid()
[[port()]] = port()
[[reference()]] = reference()
[[float()]] = float()
[[integer()]] = integer()
[[neg_integer]] = infy..-1
[[non_neg_integer]] = 0..infy
[[pos_integer]] = 1..infy
[[tuple()]] = tuple()
[[{T̄}]] = {[T̄]}
[[%{F1, ..., Fn}]] = map(⊕i=1n[[Fi]])
[[[]]] = empty_list()
[[nonempty_maybe_improper_list(T1, T2)]] = non_empty_list([[T1]], [[T2]])
[[«»]] = undefined
[[«_::n»]] = { binary() if n mod 8 = 0
               undefined otherwise
[[«_::_*n»]] = { binary() if n mod 8 = 0
               undefined otherwise
[[«_::n, _::_*n»]] = { binary() if n mod 8 = 0
                      undefined otherwise
[[ (T̄ -> T) ]] = ([[T̄]] -> [[T]])
[[ (... -> T) ]] = { fun() if [[T]] ≈ term()
                   undefined otherwise
[[n]] = n..n
[[n..n']] = n..n'

```

Figure 5: Translation in Types

3.3 Translation of Fields

Let τ range over the right-hand side type of field types, that is, τ is either T or `if_set(T)`. We consider lists whose elements are of the form $K \Rightarrow \tau$. Given such a list L we write $K \in L$ if there exist τ such that the element $K \Rightarrow \tau$ belongs to L . We denote by `Empty` the lists that maps all the non-singleton keytypes to `not_set()`.

Given two field lists L_1 and L_2 , we write $L_1 \oplus L_2$ the list obtained by adding to L_1 all the fields elements $K \Rightarrow \tau$ of L_2 such that $K \notin L_1$. Formally:

$$\begin{aligned}
L_1 \oplus [] &= L_1 \\
L_1 \oplus [K \Rightarrow \tau, L_2] &= L_1 \oplus L_2 && \text{if } K \in L_1 \\
L_1 \oplus [K \Rightarrow \tau, L_2] &= [K \Rightarrow \tau, L_1] \oplus L_2 && \text{if } K \notin L_1
\end{aligned}$$

Figure 6: Merge in Field Types

We then define $\text{map}(L)$ as the map type whose field list is L . Then we have $\llbracket \% \{F_1, \dots, F_n\} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket)$. If we want a saturated map type (i.e., a map type that specifies all the possible keytypes), then we have $\llbracket \% \{F_1, \dots, F_n\} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket \oplus \text{Empty})$.

All it remains to do is to define the translation of field types. If $K_1, \dots, K_n$ are pairwise distinct key types, we write $[\bigvee_{i=1}^n K_i \Rightarrow \tau]$ for the list $[K_1 \Rightarrow \tau, \dots, K_n \Rightarrow \tau]$. Then the translation of single field types is defined in Figure 7.:

$$\begin{aligned}
\llbracket \text{required}(T) \Rightarrow T' \rrbracket &= \begin{cases} [:k \Rightarrow \llbracket T' \rrbracket] & \text{if } \llbracket T \rrbracket \approx :k \\ [\bigvee_{i \in I} K_i \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{if } \llbracket T \rrbracket \approx \bigvee_{i \in I} K_i \text{ and } \llbracket T \rrbracket \not\approx :k \\ \text{undefined} & \text{otherwise} \end{cases} \\
\llbracket \text{optional}(T) \Rightarrow T' \rrbracket &= \begin{cases} [\bigvee_{i \in I} K_i \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{if } \llbracket T \rrbracket \approx \bigvee_{i \in I} K_i \\ \text{undefined} & \text{otherwise} \end{cases}
\end{aligned}$$

where all K_i 's in I are pairwise distinct.

Figure 7: Translation in Field Types

4 Total Approximating Translation of TypeSpec into Elixir Types

The translation defined so far is partial since some TypeSpec types do not have a corresponding Elixir type. This is not a weakness of the Elixir type system, but rather a consequence of the design choices made for Elixir types, to keep them as simple as possible while still being useful. However, it is possible to define a total translation from TypeSpec types to Elixir types that approximates the original TypeSpec type. This total translation can be useful in some contexts, for example when we want to ensure that every TypeSpec type can be represented in Elixir types, even if approximately.

More precisely, the previous translation is undefined in the following cases:

- Bitstrings with size specifications that are not a multiple of 8 bits (e.g., `<<_:3>>`).
- The type `... -> T` when the translation of T is not equivalent to `term()` (explain why this type is not interesting).
- (`n..n`)
- $\llbracket \text{required}(T) \Rightarrow T' \rrbracket$ and $\llbracket \text{optional}(T) \Rightarrow T' \rrbracket$ are undefined when $\llbracket T \rrbracket$ is not equivalent to a disjunction of pairwise distinct key types.

In the rest of this section we give a conservative extension of the translation defined so far that is total, i.e., it is defined for every TypeSpec type. The extension consists in defining approximations for the cases where the previous translation is undefined.

4.1 Approximation of Bitstring

Elixir type has no plan to support bitstring types that is finer types than `binary()`, hence, it is not possible to directly translate any bitstring types. However, most of the bitstring representatives could be trivially translated to `binary()`. According to the Erlang documentation, Bitstrings are a later generalization of binaries, so many texts and much information about binaries apply just as well for bitstrings. In other words, the purpose of bitstrings is to construct binaries in more precise manner. Elixir provides type segments as Figure 8 that specify the *unit * size* instead of

direct notation with digits. Interestingly, all the default segments are divisible by 8, that leads bitstring values to be accepted as binary types.

Segment Type = integer | float | binary | bytes | bitstring | bits | utf8 | utf16 | utf32

Figure 8: Bitstring Segment Types

Despite the rare use cases, bitstring types are still designed to be recognize *unit* and *size* other than digits divisible by 8 and these are not strictly accepted as **binary()**. These types can be addressed as **binary()** and **dynamic()**. Since Bitstrings exist to generalize binaries as mentioned above, the specification should describe a binary type and otherwise realized at compile time.

4.2 Approximation of Top Function Type